Who Guards the Agents? Review Modality, Review Intensity, and Subsequent Defect-Related Outcomes of AI Coding-Agent Pull Requests

Aditya Bhattacharjee (Student ID: 1698938)

University of Waikato

COMPX585-26B: Preparation for Research in Computer Science

Assignment 5: Research Proposal

Introduction

Problem

Artificial intelligence is becoming an important part of software development. Tools like GitHub Copilot and ChatGPT can create code, finish existing code, explain programming ideas, and help programmers find solutions. Recent studies have looked at how large language models are used in software engineering, including code creation, testing, fixing errors, writing documentation, and keeping software up to date (Hou et al., 2024).

The more AI coding tools are used, the more questions arise about how fast development is and how good the software is. AI help can make programming tasks take less time (Paradis et al., 2025), but faster code generation does not always mean better software. Research has found incomplete code, problems caused by how prompts are phrased, security issues, and problems when adding code, along with the need for programmers to check, test, and change AI-generated code (Liu et al., 2024; Mastropaolo et al., 2023; Perry et al., 2023; Zhou et al., 2025).

This problem is getting more serious as AI coding systems start to submit pull requests. Programmers still have to decide whether AI-written code is good enough to accept. How much checking is done, and how it is done, differs from project to project. This research is interested not only in whether AI can write code well but also in how AI-written code is checked before it is added to a project and whether that checking is linked to how good the code ends up being.

Motivation

Although AI tools can generate code quickly, making sure the code is correct and secure still takes a lot of work. Developers who use AI assistants often spend a lot of time checking, testing, fixing bugs, and putting together the code suggestions. The results depend a lot on the developer's experience, on how they interact with the AI tools, and on how much they trust what the tools suggest (Barke et al., 2023; Moradi Dakhel et al.,

2023; Perry et al., 2023). Developers also need to learn skills to use generative AI tools well (France, 2024).

As code written by AI becomes more common in projects, it is important to find out whether the way this code is reviewed affects the quality of the software. Reviews can vary a lot; some may look at changes carefully while others only give a quick check. Reviews done by people can be different from those done by AI tools.

This study therefore focuses on a specific question rather than the broad one of whether AI-assisted development leads to better software. The question becomes: are the review practices used on AI-generated code changes connected to the number of defects that appear later? The study will be based on real-world software projects, not on controlled coding tasks.

Background

Prior Research

AI-Assisted Software Development and Productivity

Tools like GitHub Copilot and ChatGPT help developers write code, finish existing code, find solutions, and carry out programming tasks. In a literature review, Hou et al. (2024) examined how large language models (LLMs) are used in software engineering, covering tasks such as code generation, testing, debugging, documentation, and maintenance. Their findings show that AI-assisted development extends beyond code generation to many aspects of software work. In practice, developers also use AI coding assistants for implementing features, writing tests, handling bugs, refactoring, and documentation (Vaithilingam et al., 2025).

Several studies have examined whether these tools make developers more productive. Paradis et al. (2025) ran a controlled experiment with software engineers working on a real-world enterprise task and found that tasks were completed faster when AI assistance was used. Ziegler et al. (2022) studied how developers engage with code-completion suggestions and how those interactions relate to perceived productivity.

Barke et al. (2023) identified two ways in which developers use AI coding assistants: some use them to speed up tasks they already understand, while others use them to explore solutions when a problem is unclear or complex.

These studies look at aspects of productivity. Controlled experiments can measure how fast tasks are done in a controlled environment, while studies of how developers actually use coding assistants show real-world usage. Still, neither type of evidence can prove that code will end up with fewer bugs after it is released and maintained. This shows why it is important to look at results at the repository level.

Faster code production does not always mean better software. Measuring productivity by how quickly a task is finished can overlook the extra work needed to check the code, test it, fix errors, and integrate it into the system. This is especially true when AI coding agents create changes or submit pull requests. In those cases, the effort needed to review and validate the output can be much larger than the reduction in task time would suggest.

Code Quality, Reliability and Human Oversight

Research on code generated by AI uncovered a number of quality-related issues. Nguyen and Nadi (2022) revealed that the correctness of the suggestions generated by GitHub Copilot varied depending on the programming language and the nature of the problem. Their experiment also uncovered cases of unnecessarily complicated code and reliance on helper methods that were not present in the code. Mastropaolo et al. (2023) showed that variations in phrasing could lead to different code being generated and therefore differences in the correctness of the suggestions. Finally, Liu et al. (2024) and Tosi (2024) explored the quality of code generation and emphasized the importance of assessing the results of code generation in the context of relevant tasks and quality metrics.

Another important issue pertains to the security of code that makes use of AI coding assistants. Perry et al. (2023) investigated the security properties of code written with the help of AI coding assistants. Their results showed that developers can write

insecure code believing that it is secure. Therefore, this research highlights the importance of the interactions that developers engage in with the AI coding assistant and the degree to which suggestions made by the AI are scrutinized.

Other studies have examined the effects of structured interaction and testing on the evaluation of AI-generated code. Fakhoury et al. (2024) studied a test-driven, interactive code-generation approach. This line of research underscores the need to verify AI-generated code against the specific task rather than accepting a plausible-looking solution. Developers have also reported practical problems, such as difficulties integrating AI assistants into existing development environments and compatibility issues with the code they generate (Zhang et al., 2023; Zhou et al., 2025). Overall, these studies show that code quality is influenced not only by the capabilities of the AI coding assistant but also by the extent to which a human developer participates in development and verification.

However, the majority of the studies detailed above pertain to a situation in which an AI coding assistant assists a human developer in completing a particular task during programming. The evaluation of a pull request generated by an autonomous coding agent poses a fundamentally different question: what is the process by which the pull request is evaluated, who participates in the evaluation, and what are the consequences of the outcomes of the evaluation?

Code Review and Software Quality

Code review is a well-established practice in software engineering, in which proposed changes are examined before they become part of a project. Review helps developers identify problems, discuss design choices, and decide whether changes fit the code base. McIntosh et al. (2014, 2016) found that code review coverage and participation are related to software quality in large open-source projects. Their work provides a basis for examining review practices and later quality outcomes.

However, review should not be measured only by whether a review event occurred. Two pull requests can both be approved yet differ considerably in how much scrutiny they

receive. One pull request may receive comments, requested changes, and several rounds of revision, while another may be approved with no recorded discussion. The identity of the reviewer can also matter: a pull request may be reviewed by a human developer, by an AI review tool, or by both.

These differences motivate the study of review modality and review intensity. Review modality describes who takes part in the review, and review intensity describes the amount of recorded review activity. Studying both gives a more complete picture of how pull requests are assessed than simply labelling them as reviewed or not reviewed.

Specific Contextual Details

From AI Coding Assistants to AI Coding Agents

AI-assisted software development is shifting from code suggestions within an editor to coding agents that perform development tasks and submit their changes as pull requests (PRs). A PR is a request to integrate a proposed change into a software repository. It usually records the files changed, the commits made, automated checks, reviews, and decisions about the change, such as whether it was merged.

This creates an opportunity to study the relationships between agent-generated changes, review practices, and later software quality. Where a repository records who opened and reviewed a PR, what changes were requested, when it was merged, and whether later commits reverted any of its changes, an observational study becomes possible.

The AIDev dataset (Li et al., 2025) provides such an opportunity because it contains PRs created by AI coding agents. The first step will be to determine whether its fields, repository coverage, and documentation are sufficient to identify which PRs originated from an AI coding agent and how they were reviewed. Additional repository and version-control information will be needed to analyze later changes and potential defects.

Review Modality and Review Intensity

This study will classify recorded review modality into four categories: no recorded review, AI-only review, human-only review, and hybrid review involving both human and

AI reviewers. These categories describe who participated in the recorded review process, not the quality of the review. Automated bots will be classified as AI reviewers only if their activity can reasonably be identified as code review rather than testing, formatting, or another automated task.

Review intensity will be examined separately, using indicators such as the number of review comments, review rounds, requested changes, and commits submitted after review feedback. These indicators capture different aspects of review activity. For example, a high number of review comments may indicate close scrutiny, but it may also reflect a particularly difficult change. Review intensity should therefore not be treated as a direct measure of review quality.

Each intensity indicator will initially be analyzed separately. A composite measure will be considered only if it can be properly constructed and its components are sufficiently complete and comparable. The analysis will distinguish review activity that occurs before merging from subsequent maintenance activity.

The study will also consider the characteristics of each pull request and its repository. Change size, task type, testing practices, project activity, and code complexity may be related to both review activity and later outcomes. These factors matter because differences in outcomes may not be attributable to review modality or intensity alone.

Defining Subsequent Defect-Related Outcomes

The study will examine whether merged pull requests are associated with observable defect-related events within 180 days of merging. The primary indicator is a bug-fixing commit that modifies lines introduced by the pull request, identified using an SZZ-style procedure; secondary indicators are reverts and verified follow-up fixes. Each indicator will be defined and reported separately, because these events do not provide equivalent evidence of a defect. Candidate SZZ links will be treated as defect-related outcomes rather than as proof that the pull request introduced a defect.

These signals are imperfect. A later change to a line of code does not necessarily

mean that the original change contained a defect, and version-control history does not always show where a problem originated. For this reason, the study focuses on defect-related outcomes rather than assuming that every bug fix linked to a merged pull request proves that the pull request was defective.

The analysis must also consider how often the code from a merged pull request is subsequently edited. Code that receives frequent later changes has more opportunities for a defect to be detected than code that receives little maintenance. This exposure will be taken into account when defining outcomes and interpreting results.

Research Gap

Previous research has examined the quality of AI-assisted code (Liu et al., 2024; Nguyen & Nadi, 2022), developer interaction with coding assistants (Barke et al., 2023; Perry et al., 2023), and conventional code-review practices (McIntosh et al., 2016). More recent studies of AI coding-agent pull requests have examined why such pull requests fail (Ehsani et al., 2026), how human reviewers respond to them (Watanabe et al., 2026), and how often they are reviewed, showing that many receive little or no human review (Duma et al., 2026). Projects that merge more unreviewed agent pull requests also carry a larger subsequent repair burden (Xia & Miller, 2026). However, this evidence is reported at the repository level and does not establish how the recorded review modality and intensity of an individual agent pull request are associated with its subsequent defect-related outcomes. This study will investigate those associations in public GitHub repositories and, where feasible, compare them with patterns in comparable human-authored pull requests. Because the study is observational, its findings will be interpreted as associations rather than causal effects.

The study will control for pull request and repository characteristics as far as the available data allow, and will estimate the size and uncertainty of the associations between review modality, review intensity, and defect-related outcomes.

Potential alternative explanations and the limitations of the available data will be

considered throughout the analysis and reporting.

Research Questions

This study will address four research questions:

RQ1: How do review modality and review intensity differ between AI coding-agent-authored and human-authored pull requests?

RQ2: How is review modality associated with subsequent defect-related outcomes among merged AI coding-agent-authored pull requests?

RQ3: How is review intensity associated with subsequent defect-related outcomes among merged AI coding-agent-authored pull requests that received recorded review?

RQ4: Do the associations between review practices and subsequent defect-related outcomes differ between AI coding-agent-authored and human-authored pull requests?

These questions will be addressed using data from public GitHub repositories, taking account of pull request and repository characteristics as far as the data allow.

Planned Approach

Research Design and Data Collection

This study will use a retrospective observational design based on public GitHub pull requests and repository histories. It will examine associations between recorded review practices and subsequent defect-related outcomes without manipulating review conditions or assigning reviewers experimentally.

The main study population will be pull requests authored by AI coding agents, drawn from the AIDev dataset (Li et al., 2025). The dataset will first be checked for information on pull request authors, reviewers, and review timestamps, the changes made, and the files modified. Additional information will be collected from the GitHub API or repository histories where necessary.

Whenever possible, human-authored comparison pull requests will be selected from the same repositories and time periods as the agent-authored pull requests, so that the review–outcome association can be compared across author types (RQ4). Eligibility criteria and comparison variables will be specified before any outcomes are compared. The availability of suitable comparison pull requests will be assessed during the pilot; if a sufficiently comparable group cannot be constructed, the scope of the comparison will be adjusted accordingly.

Before full data collection, a feasibility pilot will be conducted using 200 agent-authored pull requests from 20 repositories. The pilot will test whether reviewers can be identified correctly, whether review events can be reconstructed, and whether changed code can be traced through repository history. It will also identify missing information and practical problems that may require changes to the procedure.

The purpose of the pilot is to assess data availability and reliability, and whether possible defect-related outcomes can feasibly be identified. It is not intended to determine the final sample size, which will depend on eligibility, data completeness, and the frequency of outcomes.

Only merged pull requests with at least 180 days of observable repository history after merging will be included in the main study. This rule ensures that every pull request has the same opportunity for defect-related events to be observed. The final number of pull requests will depend on data availability, the pilot results, and the eligibility criteria.

Variables and Measurement

The main explanatory variables will be review modality and review intensity. Review modality will be classified into four categories according to the recorded reviewers and review activity before the pull request is merged:

- *No recorded review:* no review activity can be identified before the pull request is merged.

- *AI-only review:* recorded review activity involves only AI reviewers or review bots, with no human reviewer.
- *Human-only review:* recorded review activity involves only human reviewers, with no identifiable AI reviewer.
- *Hybrid review:* recorded review activity involves both human and AI reviewers.

These categories will be based on GitHub records, account types, and a documented classification procedure. They describe review participation and do not, in themselves, establish the quality of the review.

Review intensity will be measured using indicators such as the number of review comments relative to the size of the change, the number of review iterations, requested changes, and revisions submitted after review. Where the data permit, human and AI review activity will also be examined separately. These indicators will be analyzed individually or in combination, depending on their completeness and reliability.

The primary outcome is whether a merged pull request is associated with at least one candidate defect-related event within 180 days of merging. The main type of candidate event is a bug-fixing commit that modifies lines added by the pull request, identified using an SZZ-style procedure (Śliwerski et al., 2005). Because such procedures can link a bug fix to a change that did not cause the bug, each link will be treated as a candidate defect-related outcome rather than a definite defect-introducing change. A sample of candidate links will be inspected manually to assess their plausibility.

Secondary outcomes will include reverts, verified follow-up fixes, the number of bug-fixing changes associated with the pull request, and the time to the first defect-related event. These outcomes will be treated as indicators rather than as definitive proof that the pull request caused a defect.

The analysis will also consider characteristics that may influence both review practices and later outcomes, including the size of the pull request, the type of task, the

files changed, testing activity, repository characteristics, and the prior history of the affected code. Where feasible, the analysis will also account for later changes to the same files, because code that receives more maintenance has more opportunities for defects to be detected.

Data Analysis

The analysis will follow the four research questions presented in the Research Questions section. Each analysis will use the variables and outcome definitions described in the Variables and Measurement section.

For RQ1, descriptive statistics will compare review modality and intensity between agent-authored and human-authored pull requests, showing how often each type of review occurs and how much review activity is recorded. Regression analysis will then be used to identify which observable pull request and repository characteristics are associated with each review modality.

For RQ2, the study will examine the association between review modality and defect-related outcomes for merged agent-authored pull requests, comparing the four review categories while adjusting for observable pull request and repository characteristics. If the data allow, weighting methods based on propensity scores (Rosenbaum & Rubin, 1983) will be used to improve comparisons between review categories, and a time-to-event model will be used to examine how long it takes for the first defect-related outcome to appear during the 180-day follow-up period.

For RQ3, the study will examine whether review intensity is associated with defect-related outcomes among agent-authored pull requests that received recorded review, using measures such as comment activity, the number of review rounds, and requested changes. Interpretation will consider that intensive review may reflect more complex or riskier changes rather than review intensity itself being a direct cause.

For RQ4, agent-authored pull requests will be compared with human-authored pull requests from the same repositories and time periods, matched on characteristics such as

pull request size, task type, and code history. The analysis will examine whether the associations between review practices and subsequent outcomes differ between agent-authored and human-authored pull requests.

The primary analysis will model the binary primary outcome as a function of review practices, adjusting for other factors. The exact regression model and adjustment variables will be finalized after the pilot study, depending on how often the outcome occurs, the sample size, data completeness, and the supervisor's guidance. More complex analyses, such as propensity-score weighting and time-to-event models, will be treated as extensions rather than requirements.

Constraints and Ethical Considerations

Several limitations may affect the interpretation of the findings.

First, the study is observational, and review practices are not randomly assigned. For example, maintainers may be more likely to request human review for complex or risky changes, and these same changes may also be more likely to need later corrections. Statistical adjustment can account for observed characteristics but cannot guarantee that all important differences have been removed. The findings will therefore be interpreted as adjusted associations rather than causal effects.

Second, identifying defects from repository history is imperfect. A commit that fixes a bug does not always mean that the original pull request caused the problem. The study will document how outcomes were classified and, where possible, manually check a sample of the identified cases. Alternative outcome definitions will be used to test how sensitive the results are to these choices.

Third, recorded review activity does not always reflect review quality. An approval with no comments could reflect a thorough examination or very little attention. Similarly, an AI reviewer may identify issues without providing the same kind of oversight as a human reviewer. These differences cannot be fully captured by counting review events.

Fourth, the availability and accuracy of public GitHub data can vary. Some

human-authored pull requests may contain undisclosed AI-generated code, and some AI reviewers may be difficult to classify. The study will document these uncertainties and, where possible, manually check a sample of the reviewer classifications.

Fifth, the 180-day follow-up period will not capture defects that appear later. The findings may also not generalize to all settings, because popular public open-source projects can differ from private commercial software projects.

The study will use publicly available repository data and report results in aggregate form. No individual developers will be named in the reporting of findings. Before data collection, the researcher will confirm whether approval is required under the University of Waikato's research ethics procedures.

Evaluation of the Proposed Research

The success of the study will be evaluated in terms of methodological soundness, data completeness, measurement validity, and reproducibility, rather than whether the findings support a particular expectation.

First, the feasibility pilot will determine whether the review categories can be applied consistently and whether there is enough repository history to support the follow-up analysis. Any major gaps or classification problems will be recorded before the full analysis begins.

Second, the quality of the outcome measures will be checked by manually reviewing a sample of defect-related events. Where possible, agreement between independent raters will be reported. The analysis will also check whether the results change when different defect-identification methods or follow-up periods are used.

Third, the statistical models will be checked using appropriate diagnostics. When weighting is used, the balance of characteristics between comparison groups will be examined. If time-to-event models are used, their assumptions, such as proportional hazards, will be checked. Any serious problems will be addressed or clearly reported.

Finally, the study will provide a reproducible description of all data-processing

steps, classification rules, analysis methods, and limitations. If licensing, platform rules, and ethical guidelines allow, the analysis scripts and derived materials will be shared so that others can verify the work.

The project will be considered successful if it delivers a methodologically sound assessment of how review practices relate to later defect-related outcomes, whether the findings show a positive association, a negative association, no clear association, or different patterns across review types.

Plan

Proposed Schedule

The project is planned in stages, beginning with research-design refinement and a feasibility pilot, followed by data collection, outcome validation, statistical analysis, and dissertation writing. The schedule is provisional and may be adjusted in consultation with the supervisor, depending on data availability and the pilot results.

The proposed schedule for completing the research is presented in Table 1.

Required Resources

The project will mainly use publicly available data and open-source software. The exact resources will depend on data availability and the results of the pilot study.

Data sources. The AIDev dataset (Li et al., 2025) will be the main source of AI coding-agent pull requests. Public GitHub records, including repository and pull request histories, will be used to obtain information about reviews, code changes, and later bug fixes. Comparable human-authored pull requests will be collected from the same repositories where sufficient data are available.

Software and programming tools. Python will be used for data collection, cleaning, classification, and analysis, with libraries such as pandas for data processing and repository-mining tools for analyzing commits and source-code changes. Statistical analysis will be carried out in Python or R, depending on the methods finally selected.

Computing resources. A computer with sufficient storage and memory will be

needed to process repository data and code histories. If the data exceed local capacity, university computing facilities or cloud resources will be considered.

Research validation. Supervisor guidance will be important for refining the research questions, selecting analysis methods, and interpreting the results. A manually checked sample of pull requests and candidate defect links will be used to assess the accuracy of the classification and outcome-identification procedures.

Reproducibility requirements. The project will use publicly available data, and University of Waikato ethics requirements will be confirmed before data collection begins. Results will be reported without identifying individual developers. All data-processing decisions, analysis code, and methodological limitations will be documented so that the work can be checked and repeated, subject to data and licence conditions.

The project will not require proprietary software. Its success will depend mainly on access to repository data, the time needed to validate candidate defect links, and the computing power required for the analysis.

References

Barke, S., James, M. B., & Polikarpova, N. (2023). Grounded Copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages*, *7*(OOPSLA1), 85–111. https://doi.org/10.1145/3586030

Duma, K., Wróblewski, P., Bobińska, J., Winiarska, J., & Przymus, P. (2026). These aren't the reviews you're looking for: How humans review AI-generated pull requests. *Proceedings of the 30th International Conference on Evaluation and Assessment in Software Engineering (EASE 2026)*. https://doi.org/10.48550/arXiv.2605.02273

Ehsani, R., Pathak, S., Rawal, S., Al Mujahid, A., Imran, M. M., & Chatterjee, P. (2026). *Where do AI coding agents fail? an empirical study of failed agentic pull requests in GitHub* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2601.15195

Fakhoury, S., Naik, A., Sakkas, G., Chakraborty, S., & Lahiri, S. K. (2024). LLM-based test-driven interactive code generation: User study and empirical evaluation. *IEEE Transactions on Software Engineering*, *50*(9), 2254–2268. https://doi.org/10.1109/TSE.2024.3428972

France, S. L. (2024). Navigating software development in the ChatGPT and GitHub Copilot era. *Business Horizons*, *67*(5), 649–661. https://doi.org/10.1016/j.bushor.2024.05.009

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., & Wang, H. (2024). Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, *33*(8), Article 220. https://doi.org/10.1145/3695988

Li, H., Zhang, H., & Hassan, A. E. (2025). *AIDev: Studying AI coding agents on GitHub* [Data set]. Zenodo. https://doi.org/10.5281/zenodo.16919272

Liu, Z., Tang, Y., Luo, X., Zhou, Y., & Zhang, L. F. (2024). No need to lift a finger anymore? assessing the quality of code generation by ChatGPT. *IEEE Transactions*

on Software Engineering, *50*(6), 1548–1584. https://doi.org/10.1109/TSE.2024.3392499

Mastropaolo, A., Pascarella, L., Guglielmi, E., Ciniselli, M., Scalabrino, S., Oliveto, R., & Bavota, G. (2023). On the robustness of code generation techniques: An empirical study on GitHub Copilot. *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2149–2160. https://doi.org/10.1109/ICSE48619.2023.00181

McIntosh, S., Kamei, Y., Adams, B., & Hassan, A. E. (2014). The impact of code review coverage and code review participation on software quality: A case study of the Qt, VTK, and ITK projects. *Proceedings of the 11th Working Conference on Mining Software Repositories*, 192–201. https://doi.org/10.1145/2597073.2597076

McIntosh, S., Kamei, Y., Adams, B., & Hassan, A. E. (2016). An empirical study of the impact of modern code review practices on software quality. *Empirical Software Engineering*, *21*(5), 2146–2189. https://doi.org/10.1007/s10664-015-9381-9

Moradi Dakhel, A., Majdinasab, V., Nikanjam, A., Khomh, F., Desmarais, M. C., & Jiang, Z. M. (2023). GitHub Copilot AI pair programmer: Asset or liability? *Journal of Systems and Software*, *203*, Article 111734. https://doi.org/10.1016/j.jss.2023.111734

Nguyen, N., & Nadi, S. (2022). An empirical evaluation of GitHub Copilot's code suggestions. *Proceedings of the 19th International Conference on Mining Software Repositories (MSR 2022)*, 1–5. https://doi.org/10.1145/3524842.3528470

Paradis, E., Grey, K., Madison, Q., Nam, D., Macvean, A., Meimand, V., Zhang, N., Ferrari-Church, B., & Chandra, S. (2025). How much does AI impact development speed? an enterprise-based randomized controlled trial. *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. https://doi.org/10.1109/ICSE-SEIP66354.2025.00060

Perry, N., Srivastava, M., Kumar, D., & Boneh, D. (2023). Do users write more insecure code with AI assistants? *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2785–2799. https://doi.org/10.1145/3576915.3623157

Rosenbaum, P. R., & Rubin, D. B. (1983). The central role of the propensity score in observational studies for causal effects. *Biometrika*, *70*(1), 41–55. https://doi.org/10.1093/biomet/70.1.41

Śliwerski, J., Zimmermann, T., & Zeller, A. (2005). When do changes induce fixes? *Proceedings of the 2005 International Workshop on Mining Software Repositories*, 1–5. https://doi.org/10.1145/1082983.1083147

Tosi, D. (2024). Studying the quality of source code generated by different AI generative engines: An empirical evaluation. *Future Internet*, *16*(6), Article 188. https://doi.org/10.3390/fi16060188

Vaithilingam, P., Zhang, T., & Glassman, E. L. (2025). Using AI-based coding assistants in practice: State of affairs, perceptions, and ways forward. *Information and Software Technology*, *178*, Article 107610. https://doi.org/10.1016/j.infsof.2024.107610

Watanabe, K., Tsuchida, R., Monno, T., Huang, B., Yamasaki, K., Fan, Y., Shimari, K., & Matsumoto, K. (2026). *How AI coding agents communicate: A study of pull request description characteristics and human review responses* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2602.17084

Xia, C. S., & Miller, C. (2026). *Do these violent delights have violent ends? measuring the post-merge fate of agentic code* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2607.09902

Zhang, B., Liang, P., Zhou, X., Ahmad, A., & Waseem, M. (2023). Practices and challenges of using GitHub Copilot: An empirical study. *Proceedings of the 35th International Conference on Software Engineering and Knowledge Engineering (SEKE 2023)*, 124–129. https://doi.org/10.18293/SEKE2023-077

Zhou, X., Liang, P., Zhang, B., Li, Z., Ahmad, A., Shahin, M., & Waseem, M. (2025). Exploring the problems, their causes and solutions of AI pair programming: A study on GitHub and Stack Overflow. *Journal of Systems and Software*, *219*, Article 112204. https://doi.org/10.1016/j.jss.2024.112204

Ziegler, A., Kalliamvakou, E., Li, X. A., Rice, A., Rifkin, D., Simister, S., Sittampalam, G., & Aftandilian, E. (2022). Productivity assessment of neural code completion. *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, 21–29. https://doi.org/10.1145/3520312.3534864

Table 1

Proposed Research Schedule

Period	Activities	Expected outcome
November 2026	Refine research questions, review literature, consult supervisor, and confirm ethics requirements.	Finalized research plan.
December 2026	Conduct the pilot study and test data collection methods.	Pilot results and refined procedure.
January 2027	Collect agent-authored and comparable human-authored PR data.	Prepared dataset.
February–March 2027	Identify and validate subsequent defect-related outcomes.	Dataset with measured outcomes.
April–May 2027	Conduct statistical analyses and sensitivity checks.	Results addressing the research questions.
June–July 2027	Interpret findings and write the methodology and results chapters.	Draft chapters.
August–September 2027	Complete the discussion, limitations, conclusion, and revisions.	Complete dissertation draft.
October 2027	Proofread, check references, and prepare the final submission and presentation.	Final dissertation and presentation.

Appendix A

Preliminary Data Collection and Variable Definitions

[ht]
Table A1

Preliminary Variable Definitions

Variable	Operational definition	Data source
PR author type	AI coding agent or human author	GitHub metadata / AIDev
Review modality	No recorded review, AI-only, human-only, or hybrid review	PR review records
Review intensity	Review comments, review rounds, requested changes, and revisions after review	GitHub review history
PR size	Lines added, lines deleted, and files changed	PR metadata
Test-related indicators	Presence of tests or test changes in the PR	PR files and metadata
Primary defect-related outcome	At least one candidate defect-related event within 180 days of merging: a bug-fixing commit that modifies lines introduced by the PR (SZZ-style link)	Commit history and SZZ-style analysis
Revert indicator	Whether the PR is subsequently reverted within 180 days	GitHub commit history
Verified follow-up fix	A later commit or PR verified as fixing a problem introduced by the PR within 180 days	Commit and PR history; manual verification
Follow-up period	180 days after merging	PR and repository timestamps

Note. Note. These are operational definitions. The pilot study will check how complete the data are and whether each variable can be collected. Review categories will be used only where the available records support consistent classification.

Appendix B

Preliminary Data Collection Workflow

The proposed workflow is summarized in Table B1. It will first be applied to the pilot sample of 200 agent-authored pull requests from 20 repositories, and the steps may be revised before the main data collection.

[ht]
Table B1

Preliminary Data Collection Workflow

Step	Activity
1	Identify candidate GitHub repositories
2	Collect candidate pull requests and available metadata
3	Classify PR author type and recorded review modality
4	Apply inclusion and exclusion criteria
5	Extract review intensity measures and PR characteristics
6	Check whether each merged PR has 180 days of follow-up
7	Identify candidate defect-related outcomes, reverts, and follow-up fixes
8	Validate a sample of candidate defect links
9	Prepare the analysis dataset and document processing decisions

Quality checks will include documenting missing fields, identifying duplicate records, and manually inspecting a sample of automatically identified defect-related links. If the pilot reveals incomplete or inconsistent data, the collection and classification procedures will be revised before the main analysis.

Appendix C

Preliminary Eligibility Criteria

Proposed inclusion criteria:

- Public GitHub pull requests that have identifiable author information.
- Merged pull requests with at least 180 days of observable repository history after merging.
- Pull requests where review, change, and commit metadata can be accessed and used for evaluation.
- Pull requests authored by agents as well as comparable pull requests authored by humans, when available.

Proposed exclusion criteria:

- Pull requests that do not have data to determine the author's identity.
- Merged pull requests that do not have a 180-day observation period.
- Pull requests where necessary metadata is missing or cannot be accessed for the intended analysis.
- Duplicate records.

These criteria are provisional. The final criteria will be set after assessing the dataset and will be applied consistently to agent-authored and human-authored pull requests where comparison is intended.

Appendix D

Preliminary Statistical Analysis Plan

The analysis will begin with data-quality checks and descriptive statistics. The methods will then be finalized on the basis of what the pilot study shows about the number of eligible pull requests and how often outcomes occur (see Table D1).

The study is observational, and results will be reported as associations rather than as proof that a specific review modality or level of intensity leads to an outcome. Advanced techniques, such as propensity-score weighting and time-to-event analysis, will be applied only if the data support them and they can be completed within the project's constraints.

[ht]
Table D1

Preliminary Statistical Analysis Plan

Research component	Proposed analysis	Purpose
Describe the sample	Counts, percentages, medians, ranges, and distributions as appropriate	Summarize the sample and identify missing or unusual values
Compare review modalities	Descriptive comparisons of defect-related outcomes across feasible review categories	Identify patterns for further investigation without interpreting them as causal effects
Examine review intensity	Regression models relating review intensity measures to outcomes, if sample size and outcome counts permit	Estimate adjusted associations between review measures and subsequent outcomes
Adjust for confounding	Include relevant pull request and repository characteristics, such as PR size, test-related indicators, language, repository activity, and time period, where available	Reduce measured confounding while acknowledging that unmeasured confounding may remain
Assess robustness	Sensitivity checks using alternative outcome definitions and, where feasible, alternative model specifications	Determine whether conclusions depend strongly on analytical choices
Validate outcome links	Manually inspect a sample of candidate SZZ-style links and document disagreements or uncertainty	Assess the plausibility of automatically identified defect-related outcomes

Appendix E

Preliminary Research Ethics and Data Management

The study will use publicly available software repository data. Before data collection, the researcher will confirm the University of Waikato's applicable ethics requirements and comply with the terms and conditions of the platforms and data sources. The collected data and analysis scripts will be stored securely, and dataset versions, collection dates, filtering decisions, variable definitions, and analytical steps will be documented to ensure reproducibility.

Findings will be reported mainly in aggregate form, and the study will avoid unnecessary disclosure or ranking of individual developers and reviewers. The public availability of repository data does not in itself remove privacy, contextual, or ethical concerns, and these will be respected throughout the study.

Challenges and limitations, including restricted metadata, uncertainty about agent identity, shortcomings in the defect-linking procedure, and the limited scope of open-source repositories, will be acknowledged in the proposal and in the reported findings. The findings will not be assumed to generalize beyond the population sampled.

Note. This appendix presents a proposal, not an approved procedure. The final procedure will be determined in consultation with the University of Waikato and the supervisor.